

Data Representation

Syllabus

1.1 Number Systems

- 1 Understand how and why computers use binary to represent all forms of data
 - Any form of data needs to be converted to binary to be processed by a computer
 - Data is processed using logic gates and stored in registers
- 2 (a) Understand the denary, binary and hexadecimal number systems
 - Denary is a base 10 system
 - Binary is a base 2 system
 - Hexadecimal is a base 16 system
- (b) Convert between
 - (i) Positive denary and positive binary
 - (ii) Positive denary and positive hexadecimal
 - (iii) Positive hexadecimal and positive binary
- 3 Understand how and why hexadecimal is used as a beneficial method of data representation
 - Values used will be integers only
 - Conversions in both directions, e.g., denary to binary or binary to denary
 - Maximum binary number length of 16-bit
 - Areas within computer science that hexadecimal is used should be identified
 - Hexadecimal is easier for humans to understand than binary, as it is a shorter representation of the binary
- 4 (a) Add two positive 8-bit binary integers
- (b) Understand the concept of overflow and why it occurs in binary addition
 - An overflow error will occur if the value is greater than 255 in an 8-bit register
 - A computer or a device has a predefined limit that it can represent or store, for example 16-bit
 - An overflow error occurs when a value outside this limit should be returned
- 5 Perform a logical binary shift on a positive 8-bit binary integer and understand the effect this has on the positive binary integer
 - Perform logical left shifts
 - Perform logical right shifts
 - Perform multiple shifts
 - Bits shifted from the end of the register are lost and zeros are shifted in at the opposite end of the register
 - The positive binary integer is multiplied or divided according to the shift performed



- 6 Use two's complement to represent positive and negative 8-bit binary integers
- The most significant bit(s) or least significant bit(s) are lost
 - Convert a positive binary or denary integer to a two's complement 8-bit integer and vice versa
 - Convert a negative binary or denary integer to a two's complement 8-bit integer and vice versa

1.2 Text, Sound & Images

- 1 Understand how and why a computer represents text and the use of character sets, including American standard code for information interchange (ASCII) and Unicode
 - Text is converted to binary to be processed by a computer
 - Unicode allows for a greater range of characters and symbols than ASCII, including different languages and emojis
 - Unicode requires more bits per character than ASCII
- 2 Understand how and why a computer represents sound, including the effects of the sample rate and sample resolution
 - A sound wave is sampled for sound to be converted to binary, which is processed by a computer
 - The sample rate is the number of samples taken in a second
 - The sample resolution is the number of bits per sample
 - The accuracy of the recording and the file size increases as the sample rate and resolution increase
- 3 Understand how and why a computer represents an image, including the effects of the resolution and color depth
 - An image is a series of pixels that are converted to binary, which is processed by a computer
 - The resolution is the number of pixels in the image
 - The color depth is the number of bits used to represent each color
 - The file size and quality of the image increases as the resolution and color depth increase



1.3 Data Storage & Compression

- 1 Understand how data storage is measured
 - Including:
 - bit
 - nibble
 - byte
 - kibibyte (KiB)
 - mebibyte (MiB)
 - gibibyte (GiB)
 - tebibyte (TiB)
 - pebibyte (PiB)
 - exbibyte (EiB)
 - The amount of the previous denomination present in the data storage size, e.g.,
 - bits in a byte
 - 1024 mebibytes in a gibibyte
- 2 Calculate the file size of an image file and a sound file, using information given
 - Answers must be given in the units specified in the question
 - Information given may include:
 - Image resolution and color depth
 - Sound sample rate, resolution and length of track
- 3 Understand the purpose of and need for data compression
 - Compression exists to reduce the size of the file
 - The impact of this is, e.g.,
 - Less bandwidth required
 - Less storage space required
 - Shorter transmission time
- 4 Understand how files are compressed using lossy and lossless compression methods
 - Lossless compression reduces the file size without permanent loss of data, e.g. run length encoding (RLE)
 - Lossy compression reduces the file size by permanently removing data, e.g., reducing resolution or color depth, reducing sample rate or resolution



1.1 Number Systems

Binary Representation

The Binary Number System is the fundamental building component of every computer. Because this system only contains 1's and 0's, it was chosen. Computers may be represented using the binary system because they have countless millions of tiny "switches" that must be in the ON or OFF position. When a switch is in the ON position, it is represented by 1 and when it is in the OFF position, by 0.

Binary System

Binary Number System has only two digits i.e. 0 and 1, also known as Base 2 number system because it has only 2 digits that is why it is represented using powers of 2.

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
1	0	1	1	1	0	0	1

Denary Number System

Denary Number System uses digits from 0 – 9 and their combinations, also known as Base 10 number system. It counts in multiple of 10's such as 10, 100, 1000 and so on.

10^4	10^3	10^2	10^1	10^0
10000	1000	100	10	1
3	5	7	4	2

Binary To Denary Conversion

In order, to convert Binary to Denary we have to multiply Binary Number to the powers of 2.

For Example: $(10110111)_2 \rightarrow (?)_{10}$

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
1	0	1	1	0	1	1	1

$$1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$1 \times 128 + 0 \times 64 + 1 \times 32 + 1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 1 \times 1$$

$$128 + 0 + 32 + 16 + 0 + 4 + 2 + 1 = 183$$



+92-335-8467840



cswithsk@gmail.com

$$(10110111)_2 \rightarrow (183)_{10}$$

Denary to Binary Conversion

There are two methods to convert Denary Number to Binary Number.

Method 1

In this method, subtract the largest possible power of 2 and keep doing this until the value 0 is reached. This will give us the following 8-bit binary number:

For Example: $(152)_{10} \rightarrow (?)_2$

$$152 - 128 = 24; 24 - 16 = 8; 8 - 8 = 0$$

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
1	0	0	1	1	0	0	0

$$(152)_{10} \rightarrow (?)_2$$

Method 2

In this method we successively divide the number by 2, until the remainder is 0. Read the remainders from top to bottom.

For Example: $(173)_{10} \rightarrow (?)_2$

2	173	1
2	86	0
2	43	1
2	21	1
2	10	0
2	5	1
2	2	0
	1	

$$(173)_{10} \rightarrow (10101101)_2$$

Hexadecimal System

Hexadecimal Number System is based on 16 different digits (0 – 9, A – F), also known as Base 16. A to F are used to represent (hex) digits i.e. A = 10; B = 11, C = 12; D = 13; E = 14; F = 15.

16^4	16^3	16^2	16^1	16^0
65536	4096	255	16	1
3	2	A	C	5



+92-335-8467840



cswithsk@gmail.com

As we know, $16 = 2^4$ this means 1 hex digit is equal to 4 binary digits.

Binary	Hexadecimal	Denary
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	A
1011	11	B
1100	12	C
1101	13	D
1110	14	E
1111	15	F

Binary to Hexadecimal

To convert Binary Number to Hexadecimal Number we split the bits into groups of 4 bits, and convert it into Hexadecimal Number and if number of bits is not a multiple of 4 we can add 0's to the left.

For Example: $(101111100010)_2 \rightarrow (?)_{16}$

1	0	1	1	1	1	1	0	0	0	1	0
8	4	2	1	8	4	2	1	8	4	2	1
8 + 2 + 1 = 11				8 + 4 + 2 = 14				2			
B				E				2			

$(101111100010)_2 \rightarrow (BE2)_{16}$

Hexadecimal to Binary

For converting Hexadecimal to Binary, take each Hexadecimal digit and write down the 4-bit code corresponding to the digit.

For Example: $(D9F)_{16} \rightarrow (?)_2$

D				9				F			
8	4	2	1	8	4	2	1	8	4	2	1
1	1	0	1	1	0	0	1	1	1	1	1



+92-335-8467840



cswithsk@gmail.com

Hexadecimal to Denary

For converting Hexadecimal to Denary multiply it with the respective power of 16.

For Example: $(8BE)_{16} \rightarrow (?)_{10}$

8	B	E
16^2	16^1	16^0
256	16	1
$8 \times 256 = 2048$	$11 \times 16 = 176$	$1 \times 14 = 14$

$$2048 + 176 + 14 = 2238$$

$$(8BE)_{16} \rightarrow (2238)_{10}$$

Denary to Hexadecimal

To convert Denary Number to Hexadecimal Number successive division by 16 until 0 is reached. Remainders from bottom to top to get the Hexadecimal Number

For Example: $(2322)_{10} \rightarrow (?)_{16}$

16	2334	14 = E
16	145	1
	9	

$$(2322)_{10} \rightarrow (91E)_{16}$$

Uses of Hexadecimal System

Hexadecimal System is easier to use as 1 hex digit = 4 binary digits.

Internet Protocol (IP) Address

Each device connected to a network is given an unique address known as the Internet Protocol (IP) Address. An IPv4 address is a 32-bit number written in denary or hexadecimal form: e.g. 109.108.158.1 (or 77.76.9e.01 in hex). IPv4 has updated by the adoption of IPv6. An IPv6 address is a 128-bit number broken down into 16-bit chunks, represented by a hexadecimal number. For example: FDEC:BA98:7654:3210:ADFC:BDFF:2990:FFFF

Error Codes

Hex is often used in error messages on your computer. The hex number refers to the memory location (In computing, this is an address in the primary memory where data values are stored) of the error. This helps programmers to find and then fix problems.

Media Access Control (MAC) Address

All network adapters and network devices have a Media Access Control (MAC) address. This is also known as the 'physical address' and is a unique address determined during the



+92-335-8467840



cswithsk@gmail.com

manufacture of each device. This address is given as a set of 6 pairs of hexadecimal numbers. An example of a MAC address would be: A0-1D-48-FE-5E-F5. You can determine the physical address of the network adapters in a computer running the Windows operating system by typing the following command in to a command prompt: ipconfig/all.

Hyper Text Markup Language (HTML) Color Code

HTML is a markup language that is used for processing, definition and presentation of text like size of the text, color, bold, italic etc. It uses <tags>.

HTML is often used to represent colors of text on the computer screen. All colors can be made up of different combinations of the three primary colors (red, green and blue). The different intensity of each color (red, green and blue) is determined by its hexadecimal value. This means different hexadecimal values represent different colors using the format #RRGGBB. # symbol represents that the number written in hex form. This system uses two hex digits to represent a single color e.g. #FA6200.

There are a possible 256 values for red, 256 values for green and 256 values for blue giving a total of $256 \times 256 \times 256$ (i.e. 16 777 216) possible colors.

Addition of Binary Digits

Two Bit Addition

1 st Bit	2 nd Bit	Bit Addition	Sum	Carry
0	0	$0 + 0$	0	0
0	1	$0 + 1$	1	0
1	0	$1 + 0$	1	0
1	1	$1 + 1$	0	1

Three Bit Addition

1 st Bit	2 nd Bit	3 rd Bit	Bit Addition	Sum	Carry
0	0	0	$0 + 0 + 0$	0	0
0	0	1	$0 + 0 + 1$	1	0
0	1	0	$0 + 1 + 0$	1	0
0	1	1	$0 + 1 + 1$	0	1
1	0	0	$1 + 0 + 0$	1	0
1	0	1	$1 + 0 + 1$	0	1
1	1	0	$1 + 1 + 0$	0	1
1	1	1	$1 + 1 + 1$	1	1



+92-335-8467840



cswithsk@gmail.com

Example: Add $(11001110)_2 + (00100011)_2$

1 st no	1	1	0	0	1	1	1	0
2 nd no	0	0	1	0	0	0	1	1
Sum	1	1	1	0	1	1	0	1
Carry	0	0	0	0	0	0	1	0

Overflow

This type of addition generates an extra bit (9th bit) which shows that sum has exceeded its value and is also known as Overflow Error and also represents that this number is too large to be stored in computer using 8 bits.

Example: Add $111 + 223$ $(01101111)_2 + (11011111)_2$

1 st no		0	1	1	0	1	1	1	1
2 nd no		1	1	0	1	1	1	1	1
Sum	1	0	1	0	0	1	1	1	0
Carry		1	1	1	1	1	1	1	1

Logical Binary Shifts

Computers can carry out a logical shift on a sequence of binary numbers. The logical shift means moving the binary number to the left or to the right. Each shift left is equivalent to multiplying the binary number by 2 and each shift right is equivalent to dividing the binary number by 2. If any place is empty fill it with 0. But we can shift only to certain extent after which it will contain 0's only and that would result in the generation of error message.

Example: $(00011111)_2 = (31)_{10}$

Most Significant Bit is the left-most bit.

128	64	32	16	8	4	2	1
0	0	0	1	1	1	1	1

Now, fill the empty place left after shift is 0.

128	64	32	16	8	4	2	1
0	0	1	1	1	1	1	0

$(00111110)_2 = (62)_{10} \Rightarrow 31 \times 2^1$

Two places shift on the left

128	64	32	16	8	4	2	1
0	1	1	1	1	1	0	0



+92-335-8467840



cswithsk@gmail.com

$$(01111100)_2 = (124)_{10} \Rightarrow 31 \times 2^2$$

Three places shift on the left

128	64	32	16	8	4	2	1
1	1	1	1	1	0	0	0

$$(11111000)_2 = (248)_{10} \Rightarrow 31 \times 2^3$$

Four places shift on the left

128	64	32	16	8	4	2	1
1	1	1	1	0	0	0	0

$$(11110000)_2 = (240)_{10} \neq 31 \times 2^4$$

The left-most 1-bit has lost which means it has exceeded its limit which results in error.

Two's Complement

For representing negative numbers we take the most significant bit negative

Example: $(11000110)_2$

-128	64	32	16	8	4	2	1
1	1	0	0	0	1	1	0

$$-128 + 64 + 4 + 2 = (-58)_{10}$$

To take Two's Complement we need to invert the bits and then add '1' to LSB (Least Significant Bit).

Example: $(01111110)_2 = (126)_{10}$

Step 1: Invert all the bits

-128	64	32	16	8	4	2	1
0	1	1	1	1	1	1	0
1	0	0	0	0	0	0	1

Step 2: Add 1 to LSB

-128	64	32	16	8	4	2	1
1	0	0	0	0	0	0	1
+							1
1	0	0	0	0	0	1	0

$$(-126)_{10}$$



+92-335-8467840



cswithsk@gmail.com

1.2 Text, Sound & Images

Character Sets

Every word is made up of symbols or characters. When you press a key on a keyboard, a number is generated that represents the symbol for that key. This is called a character code. A complete collection of characters is a character set.

Text and numbers can be encoded in a computer as patterns of binary digits. Hexadecimal is a shortcut for representing binary. ASCII and Unicode are important character sets that are used as standard.

ASCII

The ASCII character set is a 7-bit set of codes that allows 128 different characters. That is enough for every upper-case letter, lower-case letter, digit and punctuation mark on most keyboards. ASCII is only used for the English language.

Extended ASCII uses 8-bit codes (0 to 255 in denary or 0 to FF in hexadecimal). This gives another 128 codes to allow for characters in non-English alphabets and for some graphical characters to be included:

dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char
0	0	000	NULL	32	20	040	space	64	40	100	@	96	60	140	'
1	1	001	SOH	33	21	041	!	65	41	101	A	97	61	141	a
2	2	002	STX	34	22	042	"	66	42	102	B	98	62	142	b
3	3	003	ETX	35	23	043	#	67	43	103	C	99	63	143	c
4	4	004	EOT	36	24	044	\$	68	44	104	D	100	64	144	d
5	5	005	ENQ	37	25	045	%	69	45	105	E	101	65	145	e
6	6	006	ACK	38	26	046	&	70	46	106	F	102	66	146	f
7	7	007	BEL	39	27	047	'	71	47	107	G	103	67	147	g
8	8	010	BS	40	28	050	(	72	48	110	H	104	68	150	h
9	9	011	TAB	41	29	051	)	73	49	111	I	105	69	151	i
10	a	012	LF	42	2a	052	*	74	4a	112	J	106	6a	152	j
11	b	013	VT	43	2b	053	+	75	4b	113	K	107	6b	153	k
12	c	014	FF	44	2c	054	,	76	4c	114	L	108	6c	154	l
13	d	015	CR	45	2d	055	.	77	4d	115	M	109	6d	155	m
14	e	016	SO	46	2e	056	:	78	4e	116	N	110	6e	156	n
15	f	017	SI	47	2f	057	/	79	4f	117	O	111	6f	157	o
16	10	020	DLE	48	30	060	0	80	50	120	P	112	70	160	p
17	11	021	DC1	49	31	061	1	81	51	121	Q	113	71	161	q
18	12	022	DC2	50	32	062	2	82	52	122	R	114	72	162	r
19	13	023	DC3	51	33	063	3	83	53	123	S	115	73	163	s
20	14	024	DC4	52	34	064	4	84	54	124	T	116	74	164	t
21	15	025	NAK	53	35	065	5	85	55	125	U	117	75	165	u
22	16	026	SYN	54	36	066	6	86	56	126	V	118	76	166	v
23	17	027	ETB	55	37	067	7	87	57	127	W	119	77	167	w
24	18	030	CAN	56	38	070	8	88	58	130	X	120	78	170	x
25	19	031	EM	57	39	071	9	89	59	131	Y	121	79	171	y
26	1a	032	SUB	58	3a	072	:	90	5a	132	Z	122	7a	172	z
27	1b	033	ESC	59	3b	073	;	91	5b	133	[	123	7b	173	(
28	1c	034	FS	60	3c	074	<	92	5c	134	\	124	7c	174	
29	1d	035	GS	61	3d	075	=	93	5d	135	]	125	7d	175	)
30	1e	036	RS	62	3e	076	>	94	5e	136	^	126	7e	176	-
31	1f	037	US	63	3f	077	?	95	5f	137	_	127	7f	177	DEL

Unicode

Unicode can represent all languages of the world, thus supporting many operating systems, search engines and internet browsers used globally. There is overlap with standard ASCII



+92-335-8467840



cswithsk@gmail.com

code, since the first 128 (English) characters are the same, but Unicode can support several thousand different characters in total. It will support up to four bytes per character.

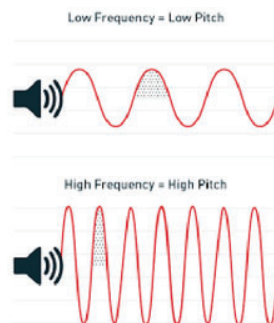
The Unicode consortium was set up in 1991. Version 1.0 was published with five goals; these were to:

- Create a universal standard that covered all languages and all writing systems
- Produce a more efficient coding system than ASCII
- Adopt uniform encoding where each character is encoded as 16-bit or 32-bit code
- Create unambiguous encoding where each 16-bit and 32-bit value always represents the same character
- Reserve part of the code for private use to enable a user to assign codes for their own characters and symbols (useful for Chinese and Japanese character sets, for example).

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+100x	က	ခ	ဂ	ဃ	င	စ	ဆ	ဇ	ဈ	ည	ဋ	ဌ	ဍ	ဎ	ဏ	တ
U+101x	ထ	ဏ	ဒ	ဓ	န	ပ	ဖ	ဗ	ဘ	မ	ယ	ရ	လ	ဝ	သ	ဟ
U+102x	ဉ	အ	ဓ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ
U+103x	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ
U+104x	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ
U+105x	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ
U+106x	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ
U+107x	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ
U+108x	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ
U+109x	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ	ဏ

Representation of Sound

Soundwaves are vibrations in the air. The human ear senses these vibrations and interprets them as sound. Each sound wave has a frequency, wavelength and amplitude. The amplitude specifies the loudness of the sound.



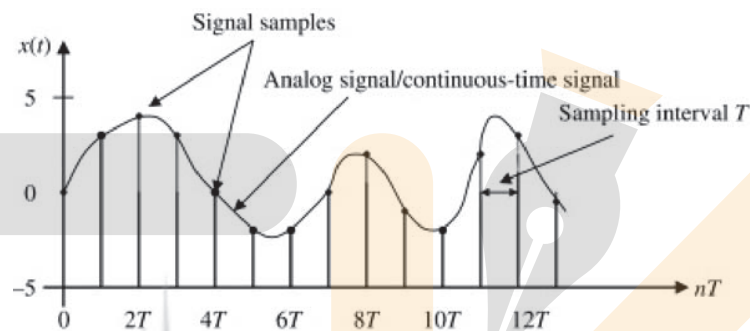
+92-335-8467840



cswithsk@gmail.com

Sound waves vary continuously. This means that sound is analogue. Computers cannot work with analogue data, so sound waves need to be sampled in order to be stored in a computer. Sampling means measuring the amplitude of the sound wave. This is done using an analogue to digital converter (ADC).

To convert the analogue data to digital, the sound waves are sampled at regular time intervals. The amplitude of the sound cannot be measured precisely, so approximate values are stored.



Sampling Resolution: Number of bit per sample. It is also known as bit depth.

Sampling Rate

It is the number of sound samples taken per second. Measured in Hertz (Hz). 1 Hz means one sample per second.

Higher the sampling rate, greater the file size.

Pros

1. Larger dynamic range
2. Better sound quality
3. Less sound distortion

Cons

1. Produces larger file size
2. Takes longer to transmit/download music files
3. Requires greater processing power

CDs have a 16-bit sampling resolution and a 44.1kHz sample rate - that is 44100 samples every second. This gives high-quality sound reproduction.

Representation of Bitmap Images

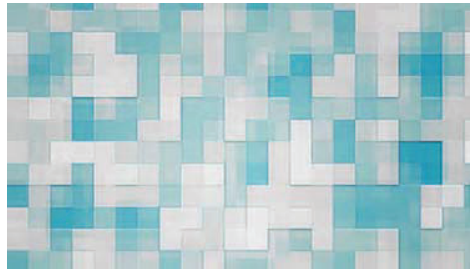
Bitmap Images are made up of Pixels (Picture elements); an image is made up of a two-dimensional matrix of a Pixel. Pixels can take different shapes such as:



+92-335-8467840



cswithsk@gmail.com



Each Pixel can be represented as Binary Number, and so bitmap image is stored in a computer as a series of binary numbers, so that:

- A black and white image only requires 1 bit per pixel – this means that each pixel can be of one of two colors, representing 1 or 0.
- If each pixel is represented by 2 bits, then each pixel can be of one of four colors ($2^2 = 4$) representing the FOUR combinations of 0 & 1 i.e. 00, 01, 10, 11.
- If each pixel is represented by 3 bits, then each pixel can be of one of eight colors ($2^3 = 8$) representing the EIGHT combinations of 0 & 1 i.e. 000, 001, 010, 011, 100, 101, 110, 111.

Color Depth

Each color is represented by number of bits. An 8-bit color depth means that each pixel can be one of 256 colors (because $2^8 = 256$). Modern computers have a 24-bit color depth, which means over 16 million different colors can be represented.

Image Resolution refers to the number of pixels that make up an image; for example, an image contains 4096×3072 pixels (12,582,912 pixels in total).

Different Screen Resolutions



Drawback of using high resolution images is the increase in file size. As the number of pixels used to represent the image is increased, the size of the file will also increase. It impacts on the time to download an image from the internet or the time to transfer images from device to device. A certain amount of reduction in resolution of an image is possible before the loss of quality becomes noticeable.



+92-335-8467840



cswithsk@gmail.com

1.3 Data Storage & Compression

Measurement of Data Storage

Bit (Binary Digit): Basic Unit of all computing memory storage terms and is either 1 or 0.

Byte: Smallest memory unit (1 byte = 8 bits).

Name of Memory Size	Equivalent Denary Values
Kilobyte (KB)	10^3 Bytes
Megabyte (MB)	10^6 Bytes
Gigabyte (GB)	10^9 Bytes
Terabyte (TB)	10^{12} Bytes
Petabyte (PB)	10^{15} Bytes
Exabyte (EB)	10^{18} Bytes

The above system of numbering now only refers to some storage devices but is technically inaccurate. It is based on the SI (base 10) system of units. 1 TB hard disk drive would allow the storage of 1×10^{12} bytes according to this system.

Memory size is measured in terms of powers of 2, another system has been adopted by the IEC (International Electrotechnical Commission) that is based on the binary system.

Name of Memory Size	Number of Bytes	Equivalent Denary Value
Kilobyte (KiB)	2^{10} Bytes	1024 Bytes
Megabyte (MiB)	2^{20} Bytes	1048576 Bytes
Gigabyte (GiB)	2^{30} Bytes	1073741824 Bytes
Terabyte (TiB)	2^{40} Bytes	1099511627776 Bytes
Petabyte (PiB)	2^{50} Bytes	1125899906842624 Bytes
Exabyte (EiB)	2^{60} Bytes	1152921504606846976 Bytes

This system is more accurate. Internal memories (such as RAM and ROM) should be measured using the IEC system. A 64GiB RAM store 64×2^{30} bytes of data (68 719476 736 bytes).

Calculation of File Size

Calculation of the file size required to hold a bitmap image and a sound sample.

File Size of Image = Image Resolution (In Pixels) $\times$ Color Depth (In Bits)

Size of Mono Sound File = Sample Rate (In Hz) $\times$ Sample Resolution in Bits $\times$ Length of Sample (In Seconds)

Stereo sound file, multiply the result by two.



+92-335-8467840



cswithsk@gmail.com

Example

A photograph is 1024×1080 pixels and uses a color depth of 32 bits. How many photographs of this size would fit onto a memory stick of 64 GB.

Multiply number of pixels in vertical and horizontal directions to find total number of pixels
 $= [1024 \times 1080] = 1\,105\,920$ Pixels.

Multiply number of pixels by color depth then divide by 8 to give the number of bytes =
 $1\,105\,920 \times 32 = 35\,389\,440/8 \text{ bytes} = 4\,423\,680 \text{ bytes.}$

$64 \text{ GB} = 64 \times 1024 \times 1024 = 68\,719\,476\,736 \text{ bytes.}$

Divide the memory stick size by the file size = $68\,719\,476\,736/4 = 423\,680 = 15\,534$ photos.

Data Compression

Sound and Image files can be sometimes very large. It is necessary to reduce (or compress) the size of a file for the following reasons:

- To save storage space on devices such as the hard disk drive/solid state drive
- To reduce the time taken to stream a music or video file » to reduce the time taken to upload, download or transfer a file across a network
- The download/upload process uses up network bandwidth - this is the maximum rate of transfer of data across a network, measured in bits per second. This occurs whenever a file is downloaded, for example, from a server. Compressed files contain fewer bits of data than uncompressed files and therefore use less bandwidth, which results in a faster data transfer rate.
- Reduced file size also reduces costs. For example, when using cloud storage, the cost is based on the size of the files stored. Internet Service Provider (ISP) may charge a user based on the amount of data downloaded.

Lossy & Lossless File Compression

Two types of File Compression: Lossy & Lossless

Lossy File Compression

This removes unnecessary data from the file. Original File cannot be retrieved after compression (loss of some details). This technique have to decide which part to keep and which one remove.

For example, Applying a lossy file compression algorithm to:

- An image, it may reduce the resolution and/or the bit/colour depth.
- A sound file, it may reduce the sampling rate and/or the resolution.



+92-335-8467840



cswithsk@gmail.com

Lossy Files are smaller in comparison to Lossless Files and benefits storage and data transfer rate.

Some of Lossy Files Compression algorithms are: MP3, MP4, JPEG

MP3

When Internet file-sharing boomed into popularity with Napster and the iPod, the MP3 cornered the market for one reason: it had a small footprint. Without broadband connections, it was impractical at the time to share file sizes larger than the MP3 standard 2 – 3 Megabytes. And that preference has stuck for some time now even though MP3 does not have nearly the same amount of quality as WAV or AIFF files. Despite this growing base of people using higher quality formats, there are still those who prefer the So, if you have a slower internet connection or limited hard drive space, MP3 could be your file format of choice. If you're worried about quality loss, don't fret too much about it. While, yes, there is a noticeable drop off in sound quality, MP3 files fall square under the "good enough" umbrella.

MP4

This format allows the storage of multimedia files rather than just sound - music, videos, photos and animation can all be stored in the MP4 format. As with MP3, this is a lossy file compression format, but it still retains an acceptable quality of sound and video. Movies, for example, could be streamed over the internet using the MP4 format without losing any real discernible quality.

JPEG

When a camera takes a photograph, it produces a raw bitmap file which can be very large in size. These files are temporary in nature. JPEG is a lossy file compression algorithm used for bitmap images. As with MP3, once the image is subjected to the JPEG compression algorithm, a new file is formed and the original file can no longer be constructed.

- Human eyes don't detect differences in color shades quite as well as they detect differences in image brightness (the eye is less sensitive to color variations than it is to variations in brightness)
- Separating pixel color from brightness, images can be split into 8x8 pixel blocks, for example, which then allows certain 'information' to be discarded from the image without causing any real noticeable deterioration in quality.

Lossless File Compression

In this type of Compression original file can be retrieved from the compressed one. This is beneficial for the crucial data, files, applications etc. None of the details is lost.



+92-335-8467840



cswithsk@gmail.com

Run Length Encoding (RLE)

Run Length Encoding (RLE) can be used for lossless compression of a number of different file formats:

RLE on Text Data

Consider the following text string: 'aaabbbbccccdddd'. Assuming each character requires 1 byte then this string needs 16 bytes. If we assume ASCII code is being used, then the string can be coded as follows:

a	a	a	b	b	b	b	c	c	c	c	d	d	d	d	d
03	97		04	98			04	99			05	100			

a = 97; b = 98; c = 99; d = 100

2nd number represents 'no of times' digit is occurring.

One Problem can be encountered with a string such as 'cdcdcdcdcd' where RLE compression isn't very effective. To cope with this, we use a flag. A flag preceding data indicates that what follows are the number of repeating units (for example, 255 05 97 where 255 is the flag and the other two numbers indicate that there are five items with ASCII code 97). When a flag is not used, the next byte(s) are taken with their face value and a run of 1 (for example, 01 99 means one character with ASCII code 99 follows).

aaaaaaaa	bbbbbbbb	c	d	c	d	c	d	eeeeeeee
08 97	10 98	01 99	01 100	01 99	01 100	01 99	01 100	08 101

The original string contains 32 characters and would occupy 32 bytes of storage.

The coded version contains 18 values and would require 18 bytes of storage.

Flag is introduced of 255 in this case produces:

255	08	97	255	10	98	99	100	99	100	99	100	255	08	101
-----	----	----	-----	----	----	----	-----	----	-----	----	-----	-----	----	-----

This has 15 values and would, therefore, require 15 bytes of storage. This is a reduction in file size of about 53% when compared to the original string.

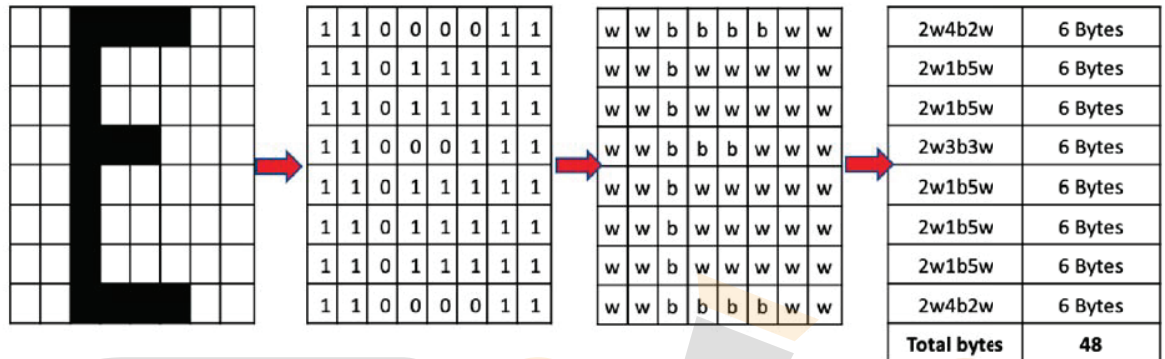


+92-335-8467840

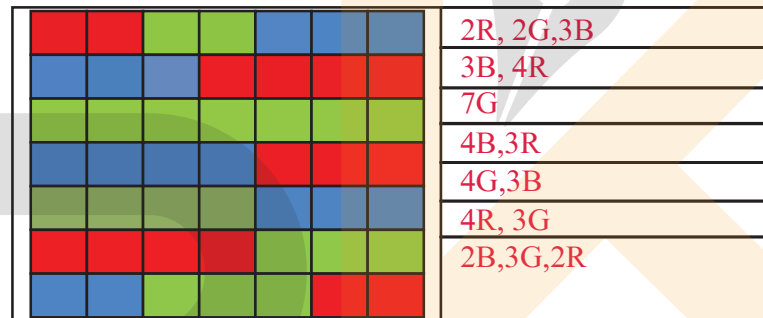


cswithsk@gmail.com

Example 1: Black & White Images



Example 2: Colored Images



The RLE code has 92 values, which means the compressed file will be 92 bytes in size. This gives a file reduction of about 52%. It should be noted that the file reductions in reality will not be as large as this due to other data which needs to be stored with the compressed file (e.g. a file header).

Sami Khatri



+92-335-8467840



cswithsk@gmail.com